Table 13. Model accuracy (0-shot, greedy generation) on ARC counterfactuals. AP = Answer Position, S = Symbol, RL = Random Letter.

Easy/Challenge	Model	AP	S	RL	AP+RL
Easy	Llama 3.1-8B	0.93	0.91	0.86	0.85
	Gemma 2-2B	0.78	0.78	0.57	0.55
	Qwen 2.5-0.5B	0.73	0.66	0.26	0.22
	GPT2-Small	0.03	0.00	0.00	0.01
Challenge	Llama 3.1-8B	0.79	0.77	0.69	0.67
	Gemma 2-2B	0.60	0.62	0.43	0.41
	Qwen 2.5-0.5B	0.56	0.51	0.18	0.19
	GPT2-Small	0.04	0.00	0.01	0.00

take the absolute value of each score, adding the highest-*magnitude* scores. Adding the highest-scoring components is more likely to yield components that perform the task well, and is better suited to the CPR metric. Adding the highest-magnitude components is more likely to yield components that have *any* strong effect on task performance, and is better suited to the CMD metric. Thus, when we report scores, we use high-value scoring to construct circuits for CPR, and high-magnitude scoring to construct circuits for CMD.

D.2. Baselines

Attribution patching. Edge attribution patching is computed as follows. Let (u, v) be an edge from component u to v, a_u and a'_u be the output activations of u on normal and counterfactual inputs, a_v be the input activations of v, and m be our model performance metric. EAP estimates the indirect effect as

$$\hat{\text{IE}} = (a_u - a'_u)\frac{\partial m}{\partial a_v}\Big|_x. \tag{3}$$

That is, we multiply the change in activation of u by the gradient (slope) of the metric m with respect to v's input, on normal inputs x.

When performing NAP (at the node level rather than edge level), the $\hat{\text{IE}}$ of a node can be computed as in Eq. 3, but replacing $\frac{\partial m}{\partial a_v}$ with $\frac{\partial m}{\partial a_u}$. Note that we always run NAP at the *neuron* granularity, and not the submodule granularity; this is because at smaller circuit sizes, including just one submodule puts us over the size threshold, meaning we have multiple points at which the circuit is empty.

Attribution patching with integrated gradients. Edge attribution patching with IG (EAP-IG) is defined as follows:

$$\hat{\text{IE}} = (a - a') \cdot \frac{1}{Z}\sum_{z=0}^{Z}\frac{\partial m(a' + \frac{z}{Z}\cdot(a - a'))}{\partial a}\Big|_x. \tag{4}$$

That is, given input x, we compute $\frac{\partial d}{\partial a}$ at Z intermediate points between a and a'. At each intermediate point, we intervene on a, replacing its activation with what it would have been at the intermediate point. Using this new activation, we recompute m, and backpropagate from that to obtain a new gradient value. We take the mean over these gradient values to obtain a more accurate estimate of the slope of m w.r.t. a. This slope is then multiplied by the change in a as before.

EAP-IG-inputs operates under a similar intuition. The key difference is that, instead of interpolating between intermediate activations at the target neuron, we only interpolate between intermediate activations at the input embeddings, and allow the network to compute the activations for the target component naturally given each intermediate input embedding. EAP-IG-activations therefore requires us to perform this interpolation for each layer separately; EAP-IG-inputs only requires us to perform this interpolation once at the inputs.

IFR. We adapt IFR to output importance scores for our computational graph as follows. Let a_v be the *input* to v; if $\mathcal{U}$ is the set of nodes with edges to v, and a_u is the *output* of a node $u \in U$, then the importance of a given u to v is

$$\text{imp}(u, v) = \frac{\max(||a_v||_1 - ||a_u - a_v||_1, 0)}{\sum_{u' \in \mathcal{U}} \max(||a_v||_1 - ||a_{u'} - a_v||_1, 0)}. \tag{5}$$

Table 14. CPR scores across circuit localization methods and ablation types. All evaluations were performed using counterfactual ablations. Higher scores are better. Arithmetic scores are averaged across addition and subtraction; see Table 17 for separate scores. We **bold** and underline the best and second-best methods per column, respectively.

	IOI				Arithmetic	MCQA			ARC (E)		ARC (C)
Method	GPT-2	Qwen-2.5	Gemma-2	Llama-3.1	Llama-3.1	Qwen-2.5	Gemma-2	Llama-3.1	Gemma-2	Llama-3.1	Llama-3.1
Random	0.25	0.28	0.30	0.25	0.25	0.27	0.32	0.26	0.32	0.26	0.25
EActP (CF)	**2.30**	1.21	-	-	-	0.85	-	-	-	-	-
EAP (mean)	0.29	0.71	0.68	0.98	0.35	0.29	0.33	0.13	0.26	0.34	0.80
EAP (CF)	1.20	0.26	1.29	0.85	0.55	0.85	1.49	1.00	1.08	<u>0.80</u>	<u>0.82</u>
EAP (OA)	0.95	0.70	-	-	-	0.29	-	-	-	-	-
EAP-IG-inputs (CF)	<u>1.85</u>	**1.63**	**3.20**	**2.08**	**0.99**	<u>1.16</u>	<u>1.64</u>	1.05	1.53	**1.04**	**0.98**
EAP-IG-activations (CF)	1.82	<u>1.63</u>	<u>2.07</u>	<u>1.60</u>	<u>0.98</u>	0.77	1.57	<u>0.79</u>	**1.70**	0.71	0.63
NAP (CF)	0.28	0.30	0.30	0.26	0.27	0.38	1.47	1.69	1.01	0.26	0.26
NAP-IG (CF)	0.76	0.29	1.52	0.42	0.39	0.77	**1.71**	**1.87**	<u>1.53</u>	0.26	0.26
IFR	0.58	0.31	0.25	0.09	0.89	0.40	0.38	0.52	0.34	0.36	0.24
UGS	0.97	0.98	-	-	-	**1.17**	-	-	-	-	-

Because important scores are normalized (for any given node, the sum of the scores of edges to it will be 1), we cannot apply a top-n procedure to find IFR circuits; we must use greedy search.

Uniform Gradient Sampling. UGS maintains a parameter $\tilde{\theta}_{(u,v)}$ for each edge (u, v), where $\theta_{(u,v)} = (1 + \exp(-\tilde{\theta}_{(u,v)}))^{-1}$ represents the estimated probability of (u, v) being part of the circuit determined by the pruning mask. The sampling frequency for $\alpha_{(u,v)}$ is determined by $w(\theta_{(u,v)}) = \theta_{(u,v)}(1 - \theta_{(u,v)})$. Specifically, $\alpha_{(u,v)} \sim \text{Unif}(0, 1)$ with probability $w(\theta_{(u,v)})$, $\alpha_{(u,v)} = 1$ with probability $\theta_{(u,v)} - \frac{1}{2}w(\theta_{(u,v)})$, and $\alpha_{(u,v)} = 0$ with probability $1 - \theta_{(u,v)} - \frac{1}{2}w(\theta_{(u,v)})$. We use $\theta_{(u,v)}$ as the importance scores when constructing circuits.

The loss function comprises two components: (1) a performance metric that measures the discrepancy between the original model's predictions and the output of the partially ablated model (here, KL divergence); and (2) a regularization term that controls the sparsity of the subgraph determined by the pruning mask. The balance between these components is governed by a hyperparameter λ. For our experiments, we set $\lambda = 10^{-3}$, chosen through a hyperparameter search over $\{10^{-2}, 10^{-3}, \ldots, 10^{-7}\}$ using a validation set. All other hyperparameters were left at their default values, as specified in Li & Janson (2024).

Optimal ablations. In optimal ablations (Li & Janson, 2024), rather than taking an activation from an example-dependent counterfactual input, we learn an ablation vector $\mathbf{a}$ that is not dependent on the original input. Given submodule u taking activations $\mathbf{u}$, we initialize the ablation vector to the mean of $\mathbf{u}$ over the task dataset $\mathcal{D}$. Then, we optimize $\mathbf{a}$ via gradient descent to minimize

$$\arg\min_{\mathbf{a}} \mathcal{L}(\mathcal{N}, \mathcal{D}, \text{do}(\mathbf{u} = \mathbf{a})), \tag{6}$$

where $\mathcal{L}$ is the cross-entropy (language modeling) loss on the task dataset $\mathcal{D}$ when we set $\mathbf{u}$ to $\mathbf{a}$. We pre-compute this vector for all u in $\mathcal{N}$, and then use these vectors as the counterfactual activations during circuit discovery.

We use initial learning rate 1×10^{-3} and batch size 20. We train for up to 1000 steps on the train split of the task dataset. We compute loss on the validation split every 50 steps; if the validation loss does not improve from its best value after 150 steps, we stop early and save the ablation vector from the best evaluation step.

D.3. Further Circuit Localization Results

Table 14 presents CPR scores for all valid task-model combinations. Trends are largely similar to those from the CMD table, except that EAP and UGS are less competitive with EAP-IG-inputs.

We provide scores for all methods where possible. UGS has significant memory requirements; running it on an 80G GPU is not possible for larger models, even when reducing the batch size to 1. Edge activation patching (EActP) and optimal ablations–based methods do not scale well time-wise with model size; the number of edges multiplies significantly, meaning

Table 15. CMD scores for the *private* test set across circuit localization methods and ablation types (lower is better). All evaluations were performed using counterfactual ablations. Arithmetic scores are averaged across addition and subtraction. We **bold** and underline the best and second-best methods per column, respectively.

	IOI				Arithmetic	MCQA			ARC (E)		ARC (C)
Method	GPT-2	Qwen-2.5	Gemma-2	Llama-3.1	Llama-3.1	Qwen-2.5	Gemma-2	Llama-3.1	Gemma-2	Llama-3.1	Llama-3.1
Random	0.75	0.72	0.70	0.75	0.75	0.73	0.68	0.74	0.68	0.73	0.75
EActP (CF)	**0.01**	0.48	-	-	-	0.35	-	-	-	-	-
EAP (mean)	0.27	0.24	0.28	0.04	0.07	0.21	0.19	0.17	0.22	0.19	0.20
EAP (CF)	0.03	0.15	0.06	**0.01**	0.01	0.12	0.08	**0.12**	**0.04**	0.20	**0.19**
EAP (OA)	0.31	0.17	-	-	-	0.11	-	-	-	-	-
EAP-IG-inp. (CF)	0.03	0.02	0.04	**0.01**	0.01	0.08	**0.06**	0.14	**0.04**	**0.10**	0.22
EAP-IG-act. (CF)	0.02	**0.01**	**0.03**	**0.01**	**0.00**	**0.05**	0.07	**0.12**	**0.04**	0.30	0.38
NAP (CF)	0.36	0.33	0.40	0.30	0.28	0.24	0.29	0.36	0.33	0.69	0.69
NAP-IG (CF)	0.25	0.19	0.29	0.18	0.17	0.18	0.29	0.33	0.27	0.67	0.67
IFR	0.43	0.70	0.75	0.89	0.22	0.60	0.62	0.49	0.66	0.63	0.53
UGS	0.04	0.02	-	-	-	-	0.19	-	-	-	-

Table 16. CPR scores for the *private* test set across circuit localization methods and ablation types. All evaluations were performed using counterfactual ablations. Higher scores are better. Arithmetic scores are averaged across addition and subtraction. We **bold** and underline the best and second-best methods per column, respectively.

	IOI				Arithmetic	MCQA			ARC (E)		ARC (C)
Method	GPT-2	Qwen-2.5	Gemma-2	Llama-3.1	Llama-3.1	Qwen-2.5	Gemma-2	Llama-3.1	Gemma-2	Llama-3.1	Llama-3.1
Random	0.25	0.28	0.30	0.25	0.25	0.27	0.32	0.26	0.32	0.26	0.25
EActP (CF)	**2.39**	1.20	-	-	-	0.87	-	-	-	-	-
EAP (mean)	0.28	0.34	0.64	0.94	0.34	0.29	0.34	0.13	0.26	0.34	0.31
EAP (CF)	1.26	0.27	1.31	0.87	0.54	0.84	1.48	1.06	1.08	0.80	0.26
EAP (OA)	1.07	0.75	-	-	-	0.29	-	-	-	-	-
EAP-IG-inputs (CF)	1.89	**1.73**	**3.03**	**2.04**	**0.98**	**1.61**	1.05	0.95	1.53	**1.05**	0.31
EAP-IG-activations (CF)	1.84	1.61	2.34	1.33	**0.98**	0.76	1.53	0.83	**1.71**	0.27	0.28
NAP (CF)	0.27	0.30	0.29	0.25	0.26	0.38	1.46	1.69	1.02	0.26	0.26
NAP-IG (CF)	0.69	0.29	1.42	0.42	0.38	0.77	**1.68**	**1.84**	1.54	0.26	0.26
IFR	0.57	0.30	0.25	0.11	0.89	0.40	0.38	0.51	0.34	0.37	**0.47**
UGS	0.97	1.00	-	-	-	1.17	-	-	-	-	-

that we must iterate over many more components. We do not include methods that take over 1 week to run.[12]

In Table 17 we provide scores for each arithmetic operator separately.

E. Details on InterpBench Model Training

Faithfulness is a fuzzy and unbound metric. Ideally, we would like to know which edges or nodes are in the circuit in advance, such that we can compute more precise metrics such as precision and recall. Inspired by InterpBench (Gupta et al., 2024), we train a transformer model closely following their methods. The model we use was explicitly trained to predict the indirect objects in the IOI dataset (App. C.1), and implements a simplified version of the IOI circuit described by Gupta et al. (2024).

The model has 6 layers and 4 heads per layer, $d_{\text{model}} = 64$, and $d_{\text{head}} = 16$. It was trained with mini-batches of varying lengths using left padding. We performed hyperparameter sweeps to find the best weights for the SIIT algorithm. We use the three-losses variant of SIIT; see Gupta et al. (2024) for details. The final model was trained for 70 hours on a single H100 GPU.

[12]This is an arbitrary threshold. We do not restrict users from submitting methods that take this long to run if they so choose.